Adaptive adversarial training under a clean protocol: what survives the audit, and what a per-norm budget does

A CIFAR-10 / PreActResNet-18 study, from the first RAMP reproduction to the norm-level controller

Matoke Brian

Status report, 15 October 2026. Draft: reviewer findings applied, not re-reviewed.

Abstract

The starting point is an MSc thesis on Adaptive Adversarial Training (AAT) that scheduled the perturbation budget ε and the attacked norm from feedback. An audit of that thesis found contradictory tables and, above all, evaluation at the adaptive ε. We re-test the idea on CIFAR-10 with a PreActResNet-18 under one protocol: AutoAttack's APGD-CE and APGD-T at *fixed* $\varepsilon = (8/255, 0.5, 12)$ for $\ell_\infty, \ell_2, \ell_1$ on the first 1000 test images, with union robustness as the per-sample AND over the three norms. Findings. (1) On the official RAMP code, per-class budget and sample reallocation did not raise union robustness or the weakest classes. (2) In our 10-epoch fine-tuning pipeline, union robustness is governed by the *total training budget*; adaptive variants sit about 1 to 2 points above the fixed-budget frontier at matched clean accuracy, but an E-AT-style non-adaptive baseline sits 3.3 above it, so that gap is not attributable to adaptivity. (3) Reallocating a roughly fixed total budget *across norms* (less on ℓ_2, more on ℓ_1) is the one large effect: about +6 union points over the frontier at 10 epochs, with at most 0.3 points lost to extra attack seeds and the full AutoAttack suite (10-epoch, seed 0). At 80 epochs from scratch the raw union gain over the fixed run recurs (26.7 to 36.8, one seed, no frontier, so not budget-adjusted). The ratio was chosen on test results and still needs held-out confirmation. (4) Automating the allocation failed: a norm-level feedback controller whose signal comes from 10-step attacks overstates ℓ_1 robustness by about 20 points, starting uniform it lands 4 points *below* the frontier, and started at the static allocation it matches it at 10 epochs and loses to it at 80 (29.6 against 36.8 union). (5) Our 80-epoch from-scratch runs do not reach official RAMP (44.4 union, 81.3 clean): the best reaches 36.8. They show monitor-only signs consistent with robust overfitting under a cosine schedule (one seed), and raising the PGD ℓ_∞ training attack from 5 to 10 steps did not change ℓ_∞ (APGD was not tested). A RAMP-inspired recipe (KL pairing, a per-batch gradient-projection variant, softplus, RAMP's learning-rate schedule) trains poorly (71.7 clean, 19.8 union), so the pipeline is not calibrated against RAMP and the remaining gap is not explained. Squint, an input-conditioned gated network, is a null result.

1 Question and starting point

The thesis proposed adaptive scheduling of the training budget (per sample or per class) and of the sampled norm, combined with a hybrid loss and KL pairing ("full AAT"). Its headline claims were never reproducible from its own tables: three different union numbers for one model, two RAMP baselines, swapped variant labels, a robust accuracy above clean accuracy, and, most importantly, strategies that evaluated each sample at its own adaptive ε. Evaluating at a smaller ε for vulnerable samples inflates robust accuracy and is not comparable to fixed-ε baselines. The audit (`docs/thesis_audit.md`, 20 items) therefore turns every thesis number into a hypothesis to re-test. The README states four: H1, full AAT beats fixed-schedule multi-norm AT on union

robustness (gain above twice the seed standard deviation, clean drop at most 1 point); H2, class-feedback ε narrows the class robustness gap; H3, adaptive norm sampling beats cyclic sampling; H4, AAT generalises better to unseen threats.

2 Protocol

Model and data. PreActResNet-18, CIFAR-10, pixels in $[0, 1]$ with normalisation inside the model. **Evaluation (never changed).** AutoAttack `standard` restricted to APGD-CE and APGD-T (the same two attacks as the official RAMP `eval.py`), fixed ε for every model, first 1000 test images, union = per-sample AND over $\ell_\infty, \ell_2, \ell_1$. The unseen-threat grid (three further budgets per norm) uses APGD-CE only. These numbers are *not* comparable to full-suite AutoAttack leaderboard numbers. Two follow-up checks are described in Section 6.1. **Training attacks.** In our own pipeline: PGD with 5, 5 and 10 steps for $\ell_\infty, \ell_2, \ell_1$ (the ℓ_1 attack is SLIDE-style, sparsity 0.95), max aggregation across norms. The official RAMP code uses 10-step APGD training attacks for ℓ_∞ and ℓ_1 only, KL pairing ($\lambda = 5$), gradient-projection fusion with a per-epoch clean model, learning rate 0.05, softplus activation, and 80 epochs from scratch. **Two pipelines.** *Official RAMP* runs (80 epochs from scratch, 790 to 860 s per epoch on a T4) form the locked baseline and the first set of experiments. *Our pipeline* fine-tunes for 10 epochs (250 to 295 s per epoch) from one shared 30-epoch ℓ_∞ pretrain, so a run costs about 45 minutes of training plus about 25 minutes of evaluation, and a pair fits on the two T4s. **The budget confound and the frontier control.** In our pipeline the training ε moves union robustness by about 21 points (Table 2), so a method that silently trains with a larger total budget looks better and loses clean accuracy. Every adaptive run is therefore compared with the fixed-ε curve (clean versus union) at matched clean accuracy, using five fixed-budget runs from 0.75× to 1.5× nominal. The frontier is a piecewise-linear chord through those points. The node slopes ($-1.3, -2.1, -1.9, -2.7$ points of union per point of clean) are not monotone, so the chord may over- or understate the true curve between nodes, and gaps below about 1 to 2 points are not resolved. It is also specific to the `v3a_max` recipe: non-adaptive recipes land up to 3 points off it (Table 2). Means are shown to one decimal; where an exact mean ends in .x5 the last digit may differ by 0.1 from other documents. **Statistics.** Two or three seeds per arm, $n = 1000$ images (binomial standard error about 1.6 points on union, about 5 per class). Differences are descriptive, not tested.

3 Phase 1: the official RAMP baseline and per-class reallocation

The official RAMP code (commit `be4971f`) reproduces thesis Table 7.1 within 1 point on every metric (two seeds). On top of it we ran a budget-neutral per-class, per-norm ε controller in two directions (`aat_rel`: classes with higher training robustness get more ε; `aat_flip`: the reverse), a class-sampling fine-tune from the locked epoch-80 weights (feedback-weighted sampler versus uniform control), and uniform averaging of the epoch 60/70/80 weights.

Findings. Neither reallocation direction raised union robustness; reversing the allocation lowered it in both seeds, and worst-class union fell in both arms, which does not support H2. The sampler and the control are within noise of each other and of locked epoch 80. Union is within 2.5 points of $\min(\ell_\infty, \ell_1)$ in every class (2-seed mean; up to 4.4 per seed) and within 1.3 points overall: the weak classes (cat, deer, dog, bird) are weak under every norm, so aligning norms has little headroom for RAMP's weights. Checkpoint averaging lowered worst-class union in both seeds (13.3 to 10.7 and 12.2 to 9.7) and changed union by -1.3 and $+2.8$. On the first 1000 train and test images, union on the weak four classes is 32.8 on train and 20.8 on test (gap 12.0), against

Table 1: Official RAMP and class-level reallocation (locked evaluation JSONs, 2-seed means, %). Worst class is the lowest per-class union.

Run	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Worst class	Union per seed
RAMP (locked baseline)	81.3	45.7	66.4	48.4	44.4	12.8	45.7, 43.1
`aat_rel`	82.9	45.8	66.9	48.6	44.7	10.8	45.2, 44.1
`aat_flip`	78.2	41.4	64.2	45.4	40.2	7.2	42.0, 38.3
control (uniform sampler)	82.5	45.0	65.4	48.5	43.5	12.8	43.6, 43.3
feedback sampler	82.7	45.7	65.8	48.6	44.6	13.3	45.2, 43.9

69.5 and 58.9 for the other six (gap 10.5); their clean accuracy on training images is 79.1 against 95.3. The weak classes are therefore also poorly fit on data the model saw, which is consistent with an optimisation- or capacity-limited explanation but does not exclude a data-limited one, and two seeds with about 100 images per class cannot separate them.

4 Phase 2: our pipeline, fixed baselines and the thesis variants

Table 2 gives the fixed-schedule runs, and Table 3 the adaptive thesis-style variants against the frontier built from them.

Table 2: Fixed-ε training budgets in our pipeline (means over seeds, %). Fraction is the multiple of the nominal training budget.

Run	Seeds	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Worst class
fixed 0.75×	3	85.9	42.3	66.7	21.3	21.1	3.1
fixed 1.0× (`v3a_max`)	3	82.9	44.6	66.2	29.9	29.3	4.2
fixed 1.125×	2	81.0	45.3	65.3	33.5	32.7	5.1
fixed 1.25×	2	79.0	47.0	64.1	37.7	36.8	5.7
fixed 1.5×	2	75.0	48.4	62.4	43.3	42.1	5.8
RAMP-style fine-tune (`ramp`)	3	82.1	45.4	65.6	30.0	29.7	5.7
E-AT-style fine-tune (`eat`)	3	83.4	43.3	66.7	32.6	31.2	7.4

Two observations. First, union rises from 21.1 to 42.1 as the budget goes from 0.75× to 1.5× while clean accuracy falls from 85.9 to 75.0: the budget confound is large, and any comparison without matching is uninformative. Second, our RAMP-style fine-tune reaches only 29.7 union against 44.4 for official RAMP, and its ℓ_1 robustness at the nominal budget is about 30 against 48.4. We suspect the ℓ_1 component of our training attack (10-step SLIDE) is weaker than RAMP's APGD ℓ_1 attack, but this is untested: the pipelines also differ in schedule length, initialisation, activation, KL pairing and gradient projection. ℓ_1 is where our models are weakest.

Findings. Raw, full AAT looks like a trade of 6.6 points of union (22.7 versus 29.3) for 4.0 points of clean accuracy against `v3a_max`; that is the budget, not the method. Against the frontier the adaptive variants whose clean accuracy lies within the node range sit +0.6 to +3.1 above it, most at +1.2 to +1.9. The E-AT-style fixed baseline sits +3.3 above the same chord (31.2 against 27.9) and the RAMP-style baseline −1.0, so the frontier describes only the `v3a_max` recipe and non-adaptive recipes move ±3 points off it. The cleanest single comparison is full AAT at 1.25× against the fixed 1.125× run, which has almost identical clean accuracy (81.1 versus 81.0): union 34.5 (seeds 34.4, 34.6) versus 32.7 (seeds 32.9, 32.5), a gap of 1.8 with non-overlapping seed ranges and $n = 2$. That is suggestive and in the right direction for H1, but it is a gap of the same

Table 3: Adaptive thesis-style variants against the fixed frontier (%). "Gap" is union minus the chord of the five fixed runs at that run's clean accuracy; `aat_full` (86.9) lies beyond the highest node (85.9), so no gap is defined.

Run	Seeds	Clean	Union	Worst class	Frontier at this clean	Gap
full AAT (`aat_full`)	3	86.9	22.7	2.9	out of range	—
full AAT, mean ε held at nominal	3	83.7	28.2	4.4	27.0	+1.2
full AAT, free per-class ε	2	83.6	28.6	4.4	27.3	+1.3
full AAT at 1.25×	2	81.1	34.5	6.1	32.6	+1.9
full AAT without ℓ_2	1	85.8	24.2	3.3	21.3	+2.9
class-ε only (`v3b_classeps`)	1	85.8	24.4	4.4	21.3	+3.1
class-ε only, nominal	1	83.1	29.3	5.6	28.7	+0.6

order as the chord error and the seed spread of the control (union range 28.7 to 30.0 over three seeds for `v3a_max`), and the README criterion is defined against `v3a_max`, `ramp` and `eat`: against those, every full-AAT variant fails either the union test or the clean-drop test (a two-seed standard deviation is not a usable noise estimate). The worst class does not improve, and E-AT, a README comparator, has a larger gap than any adaptive variant. Class-ε feedback alone (H2) shows no gain in the worst class (4.4 against a 3-seed mean of 4.2 for `v3a_max`, whose per-seed values range from 2.2 to 5.8, and 5.6 for the nominal-budget variant, one seed each: no resolvable gain). The held-out-norm test (H4) is flat: with ℓ_2 never trained on, ℓ_2 accuracy is 66.4 against 65.6 to 66.7 for the trained alternatives, consistent with ℓ_∞ training already covering ℓ_2 at 0.5. H3 (adaptive versus cyclic norm sampling) was not run.

5 Phase 3: squint, an input-conditioned network

Squint reads the perturbation it is given: a gate head routes through gated BatchNorm experts, with a no-gate ablation. Gated models need a stronger evaluation, because a saturated gate can hide gradients; we score them with the standard attacks plus extra APGD-CE attackers (gate forced to each threat, gate gradient cut, a temperature variant), per-sample AND, which is an upper bound on true robustness. Against `v3a_max` given the same number of extra attack seeds (adaptive union 28.2 and 27.4 on two seeds): squint 29.6 (clean 82.1) and the no-gate ablation 28.7 (clean 82.6). Against the chord of Table 2 (computed from the standard two-attack results) they sit 1.1 and 0.6 points below the fixed frontier (one seed each). We record squint as a null result. Its gate also does not route cleanly: the clean-input gate puts 0.6% of inputs on the first expert and the ℓ_1-adversarial gate 50%, so the attacker moves the routing, which is consistent with the gate being a weak defence.

6 Phase 4: reallocating the budget across norms

Union tracks $\min(\ell_\infty, \ell_1)$ in all of our runs, and ℓ_2 is slack: it stays between 62 and 67 whatever its training budget. At the fixed nominal budget ℓ_1 is the binding norm (about 30 against 45 for ℓ_∞), so a fixed total budget spent on ℓ_2 is largely wasted. We therefore tested a static reallocation holding ℓ_∞ at 1.0×: ℓ_2 0.75× and ℓ_1 1.5× (`realloc`), and a more extreme version with ℓ_2 0.5× and ℓ_1 1.75× (`realloc_b`).

At clean accuracy 79.6 the frontier predicts 35.7 union and `realloc` reaches 41.8, about 6 points above it, with both seeds within 0.3 of each other. It is close to the fixed 1.5× run on union (41.8

Table 4: Per-norm reallocation against the frontier (2-seed means, %). Official RAMP is the 80-epoch from-scratch result, shown for reference only.

Run	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Union per seed	Gap
fixed 1.125×	81.0	45.3	65.3	33.5	32.7	32.9, 32.5	node
`realloc` (1.0/0.75/1.5×)	79.6	42.5	65.5	50.0	41.8	41.7, 42.0	+6.2
`realloc_b` (1.0/0.5/1.75×)	78.2	41.0	64.4	54.9	41.0	41.1, 40.8	+3.1
fixed 1.5×	75.0	48.4	62.4	43.3	42.1	42.2, 42.0	node
official RAMP (80 epochs)	81.3	45.7	66.4	48.4	44.4	45.7, 43.1	—

versus 42.1) while keeping 4.6 points more clean accuracy (79.6 versus 75.0), and it is 2.6 union and 1.7 clean points below official RAMP. `realloc_b` gains less (+3.1): ℓ_∞, already the minimum norm in `realloc` (42.5 against 50.0 for ℓ_1), falls further to 41.0, union is 0.9 lower, and clean accuracy is 1.4 points lower. A hand-picked ratio cannot track which norm is the minimum, which is the argument for a controller that equalises the norms. The ratio was chosen from test-set results and `realloc` is the better of two tries, so the +6.2 is optimistically selected; confirming it on held-out test images (indices 1000 to 9999) is a prerequisite for calling it a finding. The price is robustness at larger ℓ_∞ budgets: at 16/255 `realloc` keeps 9.5% against 11.2% for fixed 1.0×, which has the same ℓ_∞ training budget (1.7 points); the comparison with 20.6% for fixed 1.5× is a budget effect. Some grid points are seen budgets for some runs (`realloc` trains ℓ_1 at 18), so the grid is not a clean unseen-threat test. The ℓ_1 curve is smooth and monotone (67.0, 38.2, 24.4 at $\varepsilon = 6, 18, 24$ against 56.8, 15.1, 5.4 for our RAMP-style run, and 66.1, 34.2, 21.6 for official RAMP from the locked files).

6.1 Is it gradient masking?

Two checks, on `realloc` and on the fixed 1.5× run as the comparison. First, five extra seeded APGD-CE runs per norm, per-sample AND with the standard result: union 41.7 becomes 41.5 for `realloc` and 42.1 becomes 41.8 for fixed 1.5× (seed 0 of each), while the plain fixed 1.0× model loses 0.9 to 1.7 points on its two seeds. Second, the full AutoAttack suite (APGD-CE, APGD-T, FAB-T, Square, the extras run only on surviving points): FAB-T removes 2 of 1000 points on ℓ_1 for `realloc` and nothing for fixed 1.5×, Square removes none, and union is unchanged (41.7 for `realloc`, 42.1 for fixed 1.5×). The extra attacks find at most 0.3 points more union on these seed-0 runs. We found no sign of masking; this is evidence, not a proof, it covers one seed, and the numbers are not comparable to the two-attack results elsewhere.

6.2 What the result does and does not say

The reallocation effect is large and survives the checks. Two limits matter. First, in our pipeline ℓ_1 at nominal ε is far below official RAMP (about 30 versus 48), which we suspect reflects a weak ℓ_1 training attack (10-step SLIDE versus RAMP's APGD ℓ_1), although the pipelines differ in other ways too. A part of the +6 may be *compensation* for that weakness rather than a general property of multi-norm AT. This is untested; the discriminating experiment is to train our pipeline with an APGD-style ℓ_1 attack, or to apply the reallocation to the official RAMP code (the per-class controllers there gave no union gain, but they did not move budget across norms). Second, it is one seed pair, one dataset and one architecture, with 10 epochs of fine-tuning from a shared pretrain.

7 Phase 5: the norm-level controller

The controller replaces the hand-picked ratio. Each epoch it measures accuracy A_p per norm at the *nominal* (fixed-evaluation) ε on a 1000-image held-out feedback split, with the same 10-step training attacks, and updates the budget fractions

$$f_p \leftarrow \mathrm{clip}\big(f_p + \alpha(\bar{A} - A_p),\, [f_{\min}, f_{\max}]\big), \qquad \tfrac{1}{|P|}\sum_p f_p = \bar{f},$$

raising ε for the weakest norm and lowering it for the strongest, with the mean fraction projected to $\bar{f} = 1.083$ (the mean of `realloc`'s fractions) so the total budget is held; $[f_{\min}, f_{\max}] = [0.5, 2]$. Measuring at the nominal ε avoids the thesis's ratchet, where reading robustness at a shrinking ε drove the budget to zero. Evaluation is unchanged and always at fixed ε. A review before the first GPU run found that resume checkpoints did not save the fractions, so a resumed run restarted the controller; this was fixed and tested (audit item 25). Two configurations: a uniform start (`normctl`, $\alpha = 0.5$) and a start at `realloc` (`normctl_ft`, $\alpha = 0.25$).

Table 5: Controller against the static allocation (seed 0 for the controllers, 2-seed mean for `realloc` at 10 epochs; %).

Schedule	Run	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Final f (ℓ_∞/ℓ_2/ℓ_1)
10 ep fine-tune	`normctl` (uniform start)	78.7	48.4	63.1	33.6	33.3	1.37 / 0.66 / 1.21
10 ep fine-tune	`normctl_ft`	78.8	45.7	64.1	46.1	42.9	1.22 / 0.55 / 1.48
10 ep fine-tune	`realloc` (static)	79.6	42.5	65.5	50.0	41.8	fixed 1.0 / 0.75 / 1.5
80 ep scratch	`normctl_ft`	76.9	44.4	60.0	29.7	29.6	1.56 / 0.50 / 1.19
80 ep scratch	`realloc` (static)	80.9	37.0	62.8	49.5	36.8	fixed

Findings. The uniform-start controller sits 4.0 points below the frontier (33.3 against 37.3 at its clean accuracy): its signal says ℓ_∞ is the weak norm (feedback accuracy, 10-step attacks on the held-out split, 0.46 against 0.52 for ℓ_1 from epoch 1 on) while AutoAttack says ℓ_1 is (33.6 against 48.4), so it moved budget to ℓ_∞ and left ℓ_1 under-trained. The 10-step feedback attack overstates ℓ_1 robustness by about 21 points (monitor 54.8 against AutoAttack 33.6 for `normctl`; 51.2 against 29.7 for the 80-epoch `normctl_ft`) and ℓ_∞ by only 2 to 4, enough to flip the ranking. This is consistent with the suspicion that the static reallocation partly compensates for a weak ℓ_1 attack, and is why this feedback signal did not find the allocation. It is not shown that the weak attack is the only cause: the accuracies of different norms are not on a common scale (audit item 25), and feedback from an APGD ℓ_1 attack was not tested. Started at the static allocation (`normctl_ft`) the controller matches it at 10 epochs: union 42.9 against 41.8 (one seed against a two-seed mean, within the 1.6-point binomial error) at 0.8 lower clean accuracy, with the norms more balanced (ℓ_∞ 45.7, ℓ_1 46.1). At 80 epochs the same controller loses clearly (29.6 against 36.8 union, 76.9 against 80.9 clean): ℓ_2 saturates at the 0.5 floor, so part of its allocation is set by the bound and not by feedback, and the feedback accuracies at epoch 79 (0.51 for ℓ_1, 0.50 for ℓ_∞) became 29.7 and 44.4 under AutoAttack. The initial point of `normctl_ft` is itself test-selected (audit item 25). We stop pursuing feedback controllers driven by the training attack.

8 Phase 6: 80 epochs from scratch, and the gap to RAMP

The raw union gain recurs, one seed each. Under our losses, moving to ℓ_2 0.75× and ℓ_1 1.5× raises union by 10.1 points at 2.7 lower clean accuracy. There is no fixed-budget frontier at 80

Table 6: 80 epochs from scratch, seed 0 (%). Rows 1 to 4 use our losses and cosine schedule; rows 5 and 6 use a RAMP-style loss and RAMP's static learning-rate schedule.

Run	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	Worst class
fixed 1.0× (`v3a_max_e80`)	83.6	39.8	63.5	27.0	26.7	3.3
`realloc`, 5-step ℓ_∞ attack	80.9	37.0	62.8	49.5	36.8	9.7
`realloc`, 10-step ℓ_∞ attack	81.7	36.5	62.8	48.7	36.3	11.1
`normctl_ft` controller	76.9	44.4	60.0	29.7	29.6	5.8
RAMP-style loss, fixed 1.0×	71.7	39.0	55.2	19.9	19.8	1.0
RAMP-style loss, ℓ_1 at 1.5×	69.9	36.7	55.6	40.1	36.1	1.9
official RAMP (2 seeds, 80 epochs)	81.3	45.7	66.4	48.4	44.4	12.8

epochs, so this is not budget-adjusted: at the 10-epoch chord slope of about -2 union points per clean point, roughly 5 of the 10.1 points could be the clean-accuracy trade. Under the RAMP-style loss the ℓ_1-1.5× run does not reallocate (it has no ℓ_2 training), it enlarges the total budget: union 19.8 to 36.1 at 1.8 lower clean accuracy.

Robust overfitting under the cosine schedule. In the monitor (10-step PGD on 500 test images, not AutoAttack) the union of the fixed and `realloc` runs peaks near epoch 49 (0.46 and 0.45) and falls to 0.41 and 0.39 by epoch 79, while ℓ_∞ training accuracy climbs to 0.82 and 0.78. AutoAttack at the end gives ℓ_∞ 39.8 and 37.0 (the `realloc` run monitored at its own training ε), against 44.6 for the same fixed budget fine-tuned for 10 epochs from a 30-epoch ℓ_∞ pretrain. This is consistent with robust overfitting [4] and with a 10-epoch fine-tune being better than 80 epochs from scratch in our pipeline; we have no checkpoints from near the peak to test it directly.

PGD step count alone does not explain the ℓ_∞ gap. Doubling the PGD ℓ_∞ training attack to 10 steps left ℓ_∞ at 36.5 against 37.0 (one seed each, a 0.5-point difference against roughly 2 points of combined noise), so step count alone does not explain the 8.7-point gap to RAMP's 45.7. APGD training was not tested, and our earlier guess that the 5-step attack was the cause is not supported.

Our RAMP-style recipe is not calibrated. With KL pairing ($\lambda = 5$), gradient projection, a softplus network, ℓ_∞ and ℓ_1 only, and the official static learning rate (0.05 until epoch 70, then 0.005), the fixed run reaches only 71.7 clean and 19.8 union, far below the official 81.3 and 44.4. Training-attack accuracy stayed low (ℓ_∞ 0.46) and the monitor was erratic until the learning-rate drop. The match is partial. What matches: $\lambda = 5$, the static schedule, ℓ_∞ and ℓ_1 only, softplus, batch size 128 and weight decay $5 \cdot 10^{-4}$. What does not: our "project" gradient projection pairs the clean and adversarial gradients of the *same* model per batch, so the update always points along the adversarial gradient and is only rescaled per tensor, whereas the official one adds a term from a separately trained clean copy once per epoch; the official attacks are 10-iteration APGD for both norms (ours PGD, 5 steps for ℓ_∞ and 10-step SLIDE for ℓ_1); the official code excludes BatchNorm and bias from weight decay; and its KL term is anchored on ℓ_∞ whereas ours pairs with the highest-loss norm. We did not isolate which of these matters. The ℓ_1-1.5× row is a larger total budget, not a reallocation. A first attempt with a cosine schedule was stopped at epoch 42 after the overfitting evidence above. We therefore cannot say what closes the gap to RAMP, and the RAMP-style rows are not evidence about RAMP's recipe.

H1	Not supported against the README comparators. Adaptive variants sit +1 to +2 above the `v3a_max` frontier at matched clean accuracy (cleanest pair +1.8, 2 seeds), but the E-AT-style baseline sits +3.3 above the same frontier, so the gap is not attributable to adaptivity; against `v3a_max`, `ramp` and `eat`, every full-AAT variant fails the union or the clean-drop test.
H2	Not supported: no worst-class gain from class-level ε feedback, on the official RAMP code or in our pipeline.
H3	Not run.
H4	Not supported: held-out ℓ_2 is flat across methods; robustness at off-nominal budgets tracks the training budget and, in the reallocation runs, shifts between norms with where the budget went (the grid overlaps some training budgets).
New	Static norm-level reallocation: about +6 union over the frontier at 10 epochs, with at most 0.3 points lost to extra attacks (10-epoch, seed 0); the raw union gain recurs at 80 epochs (one seed, not budget-adjusted). The ratio was selected on the test set, and it may partly compensate for a weak ℓ_1 training attack. A candidate finding, not yet confirmed.
New	Feedback-driven norm allocation: negative. The controller's 10-step feedback overstates ℓ_1 robustness by about 20 points; whether that is the whole cause is untested.

9 Where the hypotheses stand

10 Threats to validity

Two or three seeds for the 10-epoch comparisons and *one* seed for every 80-epoch run; 1000 test images (about 1.6 points of binomial error on union, about 5 per class); the chord frontier is coarse and recipe-specific; one dataset and one architecture; two-attack AutoAttack rather than the full suite as the headline metric (the full suite agreed on the two seed-0 runs where we checked, at 10 epochs only). Attack noise: AutoAttack in the evaluation script is unseeded, so the same checkpoint scores differently across files (for `v3a_max` seed 0, union is 30.0 in one file and 29.1 in the in-file standard run), and seeded extras cut lower-budget models by 0.9 to 1.7 against 0.2 to 0.3 for `realloc` and fixed 1.5×: the two-attack ℓ_1 evaluation is less converged for low-budget models, which is the same size as the H1 gaps. Selection: the reallocation ratio was picked from test results, and so is the starting point of the `normctl_ft` controller. The monitor numbers in Section 8 are 10-step PGD on 500 test images and are descriptive only. Pipeline: our losses and attacks do not match RAMP's, the RAMP-style reimplementation did not reproduce RAMP, and the cosine 80-epoch runs show monitor-only signs consistent with robust overfitting, so 80-epoch results in our pipeline do not transfer to RAMP. No CIFAR-100 and no thesis-scale model. Per-class conclusions are descriptive only.

11 What we would do with more compute

The weekly Kaggle quota is essentially spent. If it resets, in order of value: (1) confirm `realloc` on held-out test images (indices 1000 to 9999) and with a second seed, and run a fixed-budget frontier at 80 epochs with the static schedule; (2) apply the allocation to the *official* RAMP code, which removes our pipeline's differences, since the earlier per-class controllers there did not move budget across norms; (3) test the compensation explanation with an APGD-style ℓ_1 training attack (touches the attack code and needs the reviewer); (4) isolate why our RAMP-style reimplementation underperforms, starting with the official clean-trained-copy gradient projection; (5) save intermediate checkpoints to test robust overfitting directly; (6) test the controller with feedback from an APGD ℓ_1 attack and on a common accuracy scale.

References

[1] Matoke Brian. Adaptive Adversarial Training: A Curriculum-Based Approach to Robustness. MSc thesis (supervisor: I. Megyeri), University of Szeged, May 2025.

[2] E. Jiang and G. Singh. RAMP framework: Boosting adversarial robustness against multiple-ℓ_p perturbations. NeurIPS 2024.

[3] F. Croce and M. Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. ICML 2020.

[4] L. Rice, E. Wong and J. Z. Kolter. Overfitting in adversarially robust deep learning. ICML 2020.